

Chapter 3 Algorithm Analysis

3.1 Experimental Studies

Prompt 1: What is an experimental study of an algorithm?

Answer:

An experimental study of an algorithm involves **implementing the algorithm**, running it on **real inputs**, and **measuring how long it takes to execute**. The goal is to understand how the algorithm performs in practice by observing its actual running time.

This is done by:

- Choosing test inputs
- Running the algorithm
- Recording execution time
- Comparing results across input sizes

Experimental studies focus on **empirical performance**, not theoretical guarantees.

Prompt 2: How can running time be measured experimentally in Python?

Answer:

In Python, running time can be measured using the `time` module, specifically the `time()` function.

Example:

```
from time import time

start_time = time()

# run algorithm

end_time = time()

elapsed = end_time - start_time
```

This computes the **elapsed wall-clock time**, which represents the total real-world time that passed during execution.

Prompt 3: What is “wall-clock time,” and why can it be unreliable?

Answer:

Wall-clock time measures the total real time that passes, including:

- CPU execution
- Other running processes
- Operating system interruptions

It can be unreliable because:

- Other programs may be running simultaneously
- CPU scheduling varies
- Results may differ from one run to another

Thus, wall-clock time may not accurately reflect the algorithm’s intrinsic efficiency.

Prompt 4: Why might CPU time be a fairer metric than wall-clock time?

Answer:

CPU time measures the **actual processor time** used by the algorithm, excluding time spent waiting for other processes.

However, CPU time still has limitations:

- Resolution depends on hardware
- Repeated runs may still vary slightly
- Different machines yield different results

So while it is better, it is still not perfect.

It provides **more stable and reliable timing results** for small or fast-running code segments.

Prompt 5: Why must experiments be conducted on many input sizes?

Answer:

Because algorithm efficiency depends on **input size (n)**.

Testing many input sizes allows us to:

- Observe trends in growth
- Understand scalability
- Avoid misleading conclusions from small inputs

One input size alone tells us very little about long-term performance.

Prompt 6: How are experimental results visualized?

Answer:

Results are visualized using a **scatter plot**:

- **x-axis:** input size (n)
- **y-axis:** running time (t)

Each point represents one experiment. This helps reveal:

- Growth patterns
- Linear vs quadratic behavior
- Performance trends

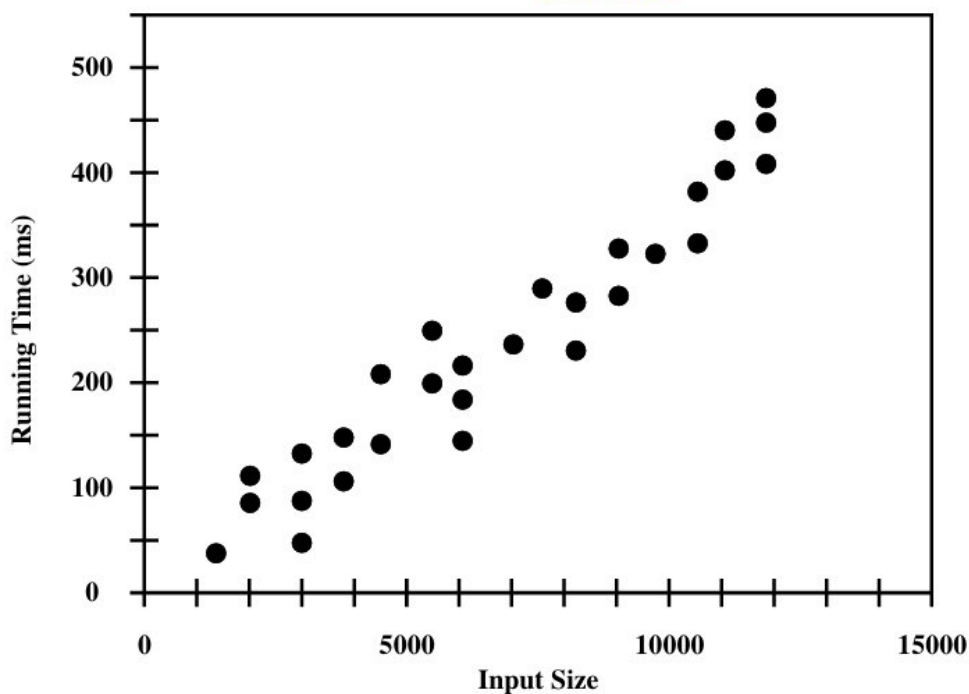


Figure 3.1: Results of an experimental study on the running time of an algorithm. A dot with coordinates (n, t) indicates that on an input of size n , the running time of the algorithm was measured as t milliseconds (ms).

Prompt 7: What is the goal of plotting running time versus input size?

Answer:

The goal is to:

- Identify how running time depends on input size
- Guess the underlying mathematical function
- Compare algorithm growth behavior

This visual intuition helps guide deeper theoretical analysis.

Prompt 8: Why are good test inputs important?

Answer:

Poor test inputs can give misleading results.

Good test inputs should:

- Cover a wide range of sizes
- Include challenging cases
- Reflect realistic usage

Without proper inputs, conclusions may be statistically invalid.

Prompt 9: What are the three major limitations of experimental analysis?

Answer:

1. **Environment dependency**
Results depend on hardware, OS, and software environment.
2. **Limited input coverage**
Only tested inputs are measured; unseen inputs may behave differently.
3. **Requires full implementation**
The algorithm must be written and debugged before testing.

Prompt 10: Why is the need for full implementation a serious drawback?

Answer:

At early design stages:

- Multiple algorithms are being considered
- Implementing all options is time-consuming
- A bad design could be rejected early with analysis

Experimental studies **cannot help before implementation**, making them inefficient for design decisions.

Prompt 11: What motivates moving beyond experimental analysis?

Answer:

We want an analysis method that:

- Works **before implementation**
- Applies to **all inputs**
- Is **independent of hardware**

This motivates **theoretical algorithm analysis**.

Prompt 12: What are the three goals of theoretical algorithm analysis?

Answer:

1. Compare algorithms independent of hardware
2. Analyze algorithms from high-level descriptions
3. Account for all possible inputs

Prompt 13: What are primitive operations?

Answer:

Primitive operations are **basic steps** that take constant time, such as:

- Assignments
- Comparisons

- Arithmetic operations
- List indexing
- Function calls and returns

They form the building blocks for time analysis.

Prompt 14: Why count primitive operations instead of measuring seconds?

Answer:

Because:

- Primitive operations are hardware-independent
- Their count correlates with actual running time
- Exact timing is unnecessary for comparison

We measure **relative efficiency**, not exact speed.

Prompt 15: What assumption is made when counting primitive operations?

Answer:

The assumption is that:

- All primitive operations take roughly the same constant time
- Differences average out over many operations

This makes counting a reasonable approximation.

Prompt 16: What is $f(n)$ in algorithm analysis?

Answer:

$f(n)$ is a function that represents:

- The number of primitive operations
- As a function of input size n

It captures the **order of growth** of an algorithm's running time.

Prompt 16: Why is worst-case analysis preferred?

Answer:

Worst-case analysis:

- Is easier than average-case analysis
- Requires no probability distributions
- Guarantees performance on all inputs

Designing for the worst case leads to **robust algorithms**.

Prompt 17: Why is average-case analysis difficult?

Answer:

Because it requires:

- A probability distribution over inputs
- Advanced probability theory
- Assumptions that may not hold in practice

Thus, it is rarely used in introductory algorithm analysis.

Prompt 18: Why does worst-case analysis lead to better algorithms?

Answer:

Because:

- If an algorithm performs well in the worst case, it performs well in all cases
- It encourages stronger, more reliable designs

Section 3.2 – The Seven Functions Used in Algorithm Analysis

Prompt 1: Why does the book focus on only seven functions for algorithm analysis?

Answer:

The book emphasizes **seven fundamental functions** for analyzing algorithms because they represent the most common patterns of algorithm growth. Focusing on these functions simplifies understanding and comparing algorithms.

Reasons:

Most algorithms fall into these categories:

- Simple algorithms (like summing a list) grow **linearly**.
- Recursive algorithms or divide-and-conquer often grow **logarithmically or quadratically**.
- Rarely, algorithms may grow cubically, exponentially, or factorially.
- These seven functions cover nearly all practical cases.

They capture order of growth (asymptotic behavior):

- Exact running times depend on hardware, compiler, or implementation.
- **Big-O analysis** focuses on how runtime grows as input size increases.
- These functions summarize growth patterns clearly.

Simplifies comparison between algorithms:

- Instead of analyzing every algorithm separately, we can classify it using one of the seven functions.
- This allows quick reasoning about efficiency and scalability.

The Seven Functions Commonly Used:

Function	Growth Pattern	Example Use Case
1	Constant	Accessing an array element
$\log n$	Logarithmic	Binary search
n	Linear	Summing elements in a list
$n \log n$	Linearithmic	Merge sort, Heap sort
n^2	Quadratic	Bubble sort, selection sort

n^3	Cubic	Some matrix operations
2^n	Exponential	Recursive Fibonacci (naive)

Prompt 2: What is the constant function, and what does it represent in algorithm analysis?

Answer:

The **constant function** is defined as:

$$f(n) = c$$

where c is a fixed constant.

- The most common constant function is:
- $f(n) = 1$

What it represents:

- Operations that take the **same amount of time regardless of input size**
- Examples:
 - Assigning a value to a variable
 - Adding two numbers
 - Accessing an array element by index

In Big-O notation:

$$O(1)$$

This is the **ideal** running time.

Prompt 3: Why is the logarithmic function so common in algorithm analysis?

Answer:

1. General Definition:

$$f(n) = \log_b n \text{ where } b > 1$$

- Here, b is the base of the logarithm.
- It tells you **how many times you need to multiply b to get n** .
- Example: $\log_2 8 = 3$ because $2^3 = 8$.

In Computer Science:

- Base b is almost always **2**, because computers work in **binary**.
- So, we usually write:

$$\log n = \log_2 n$$

- This represents **the number of times you can divide n by 2** until you reach 1.

Why it appears often:

- Many algorithms **repeatedly divide the problem in half**
- Computers use **binary representation**
- Examples:
 - Binary search
 - Balanced trees
 - Divide-and-conquer algorithms

Key intuition:

Logarithmic growth is **very slow**, even for large inputs.

Prompt 4: How can logarithms be understood without calculus?

Answer:

For integers, logarithms can be understood as:

The number of times you can divide n by the base before reaching 1

Example:

- $\log_2 12 = 4$ because:
- $12 \rightarrow 6 \rightarrow 3 \rightarrow 1.5 \rightarrow 0.75$
- **4 levels** $\rightarrow$ corresponds to $\lceil \log_2 12 \rceil = 4$

This interpretation is often used in **algorithm analysis**.

Prompt 5: What are the most important logarithm rules used in algorithm analysis?

Answer:

Key rules (Proposition 3.1):

1. $\log_b(ac) = \log_b a + \log_b c$
2. $\log_b(a/c) = \log_b a - \log_b c$
3. $\log_b(a^c) = c \log_b a$
4. $\log_b a = \log_d a / \log_d b$
5. $b^{\log_d a} = a^{\log_d b}$

Why they matter:

- Help simplify running-time expressions
- Allow base changes
- Used to compare growth rates

Prompt 6: What is the linear function and when does it occur?

Answer:

The **linear function** is:

$$f(n) = n$$

When it occurs:

- One operation per input element
- Examples:
 - Scanning a list
 - Reading input
 - Finding a maximum value

Key insight:

You **cannot do better than linear time** if every element must be examined.

Big-O notation:

$$O(n)$$

Prompt 7: What is the n-log-n function and why is it important?**Answer:**

The **n-log-n** function is:

$$f(n) = n \log n$$

Why it is important:

- It is the **best possible running time for comparison-based sorting**
- Faster than quadratic, slower than linear

Examples:

- Merge sort
- Heap sort
- Quick sort (average case)

Key takeaway:

Algorithms with $O(n \log n)$ are considered **efficient and scalable**.

Prompt 8: What causes quadratic running time in algorithms?**Answer:**

The **quadratic function** is:

$$f(n) = n^2$$

Common causes:

- Nested loops
- Comparing all pairs of elements

Example:

```
for i in range(n):  
    for j in range(n):  
        do_something()
```

Big-O notation:

$$O(n^2)$$

Quadratic algorithms become **slow quickly** as input size grows.

Prompt 9: What is the cubic function and when does it appear?

Answer:

The **cubic function** is:

$$f(n) = n^3$$

Where it appears:

- Triple nested loops
- Some matrix algorithms
- Certain brute-force solutions

Big-O notation:

$$O(n^3)$$

Cubic algorithms are **rarely practical** for large inputs.

Prompt 10: How do the seven functions compare in growth rate?

Answer:

From **slowest to fastest growth**:

1. Constant $\rightarrow 1$
2. Logarithmic $\rightarrow \log n$
3. Linear $\rightarrow n$
4. N-log-n $\rightarrow n \log n$
5. Quadratic $\rightarrow n^2$
6. Cubic $\rightarrow n^3$
7. Exponential $\rightarrow a^n$

When comparing algorithmic growth rates, we consider how fast each function increases as the input size n becomes very large.

From slowest to fastest growth:

1. **Constant:**

$$O(1)$$

- Running time does **not depend on input size**
- Example: accessing an array element

2. **Logarithmic:**

$$O(\log n)$$

- Grows very slowly
- Typical of divide-and-conquer algorithms
- Example: binary search

3. **Linear:**

$$O(n)$$

- Time increases proportionally with input size
- Example: scanning a list

4. **Linearithmic ($N \log N$):**

$$O(n \log n)$$

- Slightly faster growth than linear
- Typical of efficient sorting algorithms
- Example: merge-sort, heap-sort

5. **Quadratic:**

$$O(n^2)$$

- Growth increases rapidly
- Common in nested loops
- Example: bubble sort, selection sort

6. **Cubic:**

$$O(n^3)$$

- Very steep growth
- Often impractical for large inputs
- Example: naive matrix multiplication
-

7. Exponential:

$$O(a^n)$$

- Growth explodes extremely fast
- Becomes infeasible even for small n
- Example: brute-force solutions to NP problems

Section 3.3 – Asymptotic Analysis

Prompt 1: What is asymptotic analysis, and why do we use it in algorithm analysis?

Answer:

Asymptotic analysis is the study of how an algorithm's **running time grows as the input size n becomes large**.

We use it because:

- Exact running times depend on **hardware, programming language, compiler, and implementation**
- These details change, but **growth behavior does not**
- We care about **scalability**, not small constant differences

Asymptotic analysis lets us:

- Focus on the **dominant term** that controls growth
- Ignore **constant factors** and **lower-order terms**
- Compare algorithms **fairly and mathematically**

Key idea:

We study the *big picture* of performance, not the exact number of CPU instructions.

Prompt 2: Why do we ignore constant factors in asymptotic analysis?

Answer:

Each line of code may execute:

- A few machine instructions

- Or many, depending on the environment

Counting exact operations is:

- Complicated
- Non-portable
- Often misleading

Instead, we:

- Count **primitive operations**
- Measure performance **up to a constant factor**

For example:

- $5n$ and $100n$ both grow linearly
- Their difference matters only for **small n**

As $n \rightarrow \infty$, constants become insignificant compared to growth rate.

Prompt 3: What does it mean to characterize running time as a function of input size?

Answer:

We express running time as a **function** $f(n)$, where:

- n measures the **size of the input**
- $f(n)$ measures **how many steps the algorithm performs**

Examples of input size:

- Number of elements in a list
- Number of vertices in a graph
- Length of a string

This allows us to:

- Predict performance
- Compare algorithms
- Reason mathematically

Prompt 4: What is Big-Oh notation, formally?

Answer:

We say that

$$f(n) \text{ is } O(g(n))$$

if there exist **constants** $c > 0$ and $n_0 \geq 1$ such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

What it means in words:

1. **f(n)** is the function you are analyzing (like the runtime of an algorithm).
2. **g(n)** is a "standard" function that represents an upper bound on the growth of **f(n)** (like n , n^2 , $\log n$ etc.).
3. **c** is a constant that scales $g(n)$ so that it eventually exceeds $f(n)$ for large enough n .
4. **n₀** is the point beyond which this inequality holds.

In simple terms:

- Big-O gives an **upper bound** on how fast a function grows **as n becomes large**.
- It ignores **constants** and **small n behavior**, focusing only on **asymptotic growth**.

Example:

Suppose $f(n) = 3n + 5$.

- We want to show $f(n) = O(n)$.
- Choose $g(n) = n$.
- Pick $c = 4$ and $n_0 = 5$.

Check:

$$f(n) = 3n + 5 \leq 4n = c \cdot g(n) \text{ for all } n \geq 5$$

✓ Works! So $3n + 5 = O(n)$.

Prompt 5: How is Big-Oh used to analyze algorithm running time?

Answer:

We:

1. Identify the **input size** n
2. Count primitive operations
3. Express the total as a function of n
4. Keep only the **dominant growth term**

Example:

- If an algorithm does $3n + 20$ steps
- Its running time is **$O(n)$**

Prompt 6: Why does the `find_max` algorithm run in **$O(n)$** time?

Answer:

The algorithm:

- Initializes one variable $\rightarrow O(1)$
- Loops once through the list $\rightarrow n$ iterations
- Each iteration does:
 - One comparison
 - Possibly one assignment

Each iteration is **constant time**, and there are **n iterations**.

Thus:

$$Total\ time = c + c'n = O(n)$$

Prompt 7: What does Big-Oh allow us to ignore in running-time expressions?

Answer:

Big-Oh allows us to ignore:

- Constant multipliers
- Lower-order terms

Example:

$$5n^4 + 3n^3 + 2n^2 + 4n + 1 = O(n^4)$$

Because:

- n^4 grows faster than all other terms
- Eventually it dominates completely

Prompt 8: Why does the highest-degree term dominate a polynomial?

Answer:

As $n \rightarrow \infty$:

- n^d grows much faster than n^{d-1}, n^{d-2}

Proposition 3.9 states:

Any polynomial of degree d is $O(n^d)$

Thus:

- Growth rate depends only on the **degree**, not coefficients

Prompt 9: How do logarithmic terms compare to linear terms?

Answer:

For all $n \geq 1$:

$$\log n \leq n$$

This means **logarithmic growth is always slower than linear growth**.

Consequences:

- If you multiply a logarithmic term by n , like $n \log n$, it **grows slower than n^2** for large n :

$$\frac{n \log n}{n^2} \text{ as } n \rightarrow \infty$$

- Similarly, for large n , any expression like $5n^2 + 3n \log n$ can be simplified using Big-O notation:

$$5n^2 + 3n \log n = O(n^2)$$

because n^2 dominates $n \log n$ as n gets large.

Prompt 10: What does it mean to describe a function in “simplest terms”?

Answer:

It means choosing the **tightest** Big-Oh bound.

Bad:

- $4n^3 + 3n^2 = O(n^5)$

Good:

- $4n^3 + 3n^2 = O(n^3)$

Big-Oh should be **accurate**, not just technically correct.